NASUS

# 一种基于持续演进系统画像的变更驱动集成验证与发布准入方法

Nasus 专利 Wiki

V2.1 内部 Idea 评审版

2026年10月13日

**核心目标**

在连续代码变更下，复用并更新系统画像，确定本轮必须验证的集成行为，以目标版本的有效执行证据判断发布准入。

# 目录

一 背景与技术问题 . . . . . 3

二 技术方案设计与实现 . . . . . 5

1. 总体闭环与核心对象 . . . . . 5

2. S101 建立基准画像 . . . . . 6

3. S102 接收变化并形成新修订 . . . . . 6

4. S103 传播影响并判定失效 . . . . . 7

5. S104 确定本轮验证义务 . . . . . 8

6. S105 复用及补充验证资产 . . . . . 9

7. S106 固定计划并执行 . . . . . 11

8. S107 计算发布准入 . . . . . 12

9. S108 回流并形成下一轮画像 . . . . . 13

10. 关键技术难点与处理机制 . . . . . 14

11. 当前实现情况 . . . . . 14

12. 订单退款两轮变更实施例 . . . . . 16

三 技术保护点 . . . . . 19

四 技术效果与业务效果 . . . . . 20

参考资料 . . . . . 21

# 一 背景与技术问题

## 1. 高频代码变更与系统级验证之间的缺口

AI Coding 缩短了代码生成和修改周期，但系统理解、跨模块验证和上线判断仍需要大量人工工作。局部实现和单元测试通过后，开发者仍需确认：此次变化影响哪些接口、数据和业务链，哪些历史测试仍可使用，以及现有运行结果是否足以支持当前版本上线。

以退款为例，从全额退款增加部分退款，看似只是处理函数和参数变化，实际可能同时影响支付请求、订单状态、累计退款金额、并发控制和权限校验。随后改为异步回调，又引入消息重放、乱序和重复记账风险。仅检查接口返回成功，无法证明这些模块协同后的行为正确。

连续变更使现有验证方式暴露出四类问题：

| 问题 | 形成原因 | 对集成验证和发布判断的影响 |
| --- | --- | --- |
| 历史系统认知失效 | 代码、需求、测试和运行记录分别变化，既有关系与审阅结论缺少修订约束。 | 沿用过期入口、规则或依赖，后续分析建立在错误依据上。 |
| 变更影响遗漏 | 只检查修改文件或历史检索命中，忽略新增文件、空召回、跨模块数据及消息关系。 | 看似无关的集成行为没有进入验证范围。 |
| 重复回归成本高 | 每轮重新理解系统、生成用例或执行全部测试；直接裁剪又可能遗漏必要行为。 | 验证反馈滞后于开发，速度与验证充分性难以同时保持。 |
| 旧证据支撑新版本放行 | 测试资产、通过结论和运行目标混用，发布前没有重新检查适用性。 | 旧构建结果、不同批次成功项或不完整观察被计为当前版本通过。 |

## 2. 核心技术思路

本方案将系统画像作为可持续维护的系统认知和验证依据。画像包含代码实体、业务规范、依赖关系、测试资产及执行证据的版本关联；每次变化产生新修订，单轮分析、执行与准入使用冻结的输入。

**基准画像 → 来源变化 → 影响与失效范围 → 本轮必验义务 → 验证资产与执行计划 → 目标版本证据 → 发布准入 → 画像演进。**

关键连接在于：来源变化决定哪些历史依据失效、哪些行为必须重验；生成与选测保持这些义务；执行证据与义务及目标版本对应；准入前再次检查依赖；经确认的反馈进入下一轮画像。由此减少重复理解、重复生成和无关回归，同时保持本轮必验集合完整。

AI Coding 是主要应用场景，人工提交的代码变更也使用同一处理链。

## 3. 现有技术与本方案的技术差异

| 现有方向及资料 | 已有机制与解决范围 | 本方案的技术差异 |
| --- | --- | --- |
| 依赖图选择性回归、Predictive Test Selection。R1R2 | 将代码差异、依赖图、历史覆盖和测试结果用于选测，减少执行范围。 | 将查询范围变化纳入依据失效；影响分析输出固定必验集合，排序和预算不能删除集合中的义务。 |
| 交付质量门禁。R3 | 交付事件触发测试，结合环境、结果判定和拦截机制控制交付。 | 准入逐项消费画像变化形成的义务及目标构建证据，并在提交前重新检查版本适用性。 |
| in-toto 证据与产物流转。R4 | 通过步骤证据和工件关系验证供应链过程及产物完整性。 | 将业务验收判据、运行观察和义务满足情况纳入证据接纳条件，连接到集成验证与发布判断。 |
| 软件资料及版本化检索。R5 | 多源资料、实体关系、版本基线、增量维护、证据检索与生成校核。 | 画像修订继续驱动验证依据失效、必验范围变化、实际执行及下一轮反馈。 |
| 需求和历史用例辅助生成。R6 | 需求分片、图谱及历史用例辅助生成和评价候选测试。 | 生成用于补充缺失资产；完整义务、独立判据和执行证据分别控制生成覆盖与发布准入。 |
| 测试判据研究。R7 | 研究规格、模型和人工等正确性判据来源及其判定困难。 | 将独立批准的验收修订与每轮义务、用例、执行观察及准入决定连续绑定。 |

本方案的技术保护主题集中在这些环节之间的数据约束和状态转换，具体内容见第三部分。

# 二 技术方案设计与实现

## 1. 总体闭环与核心对象

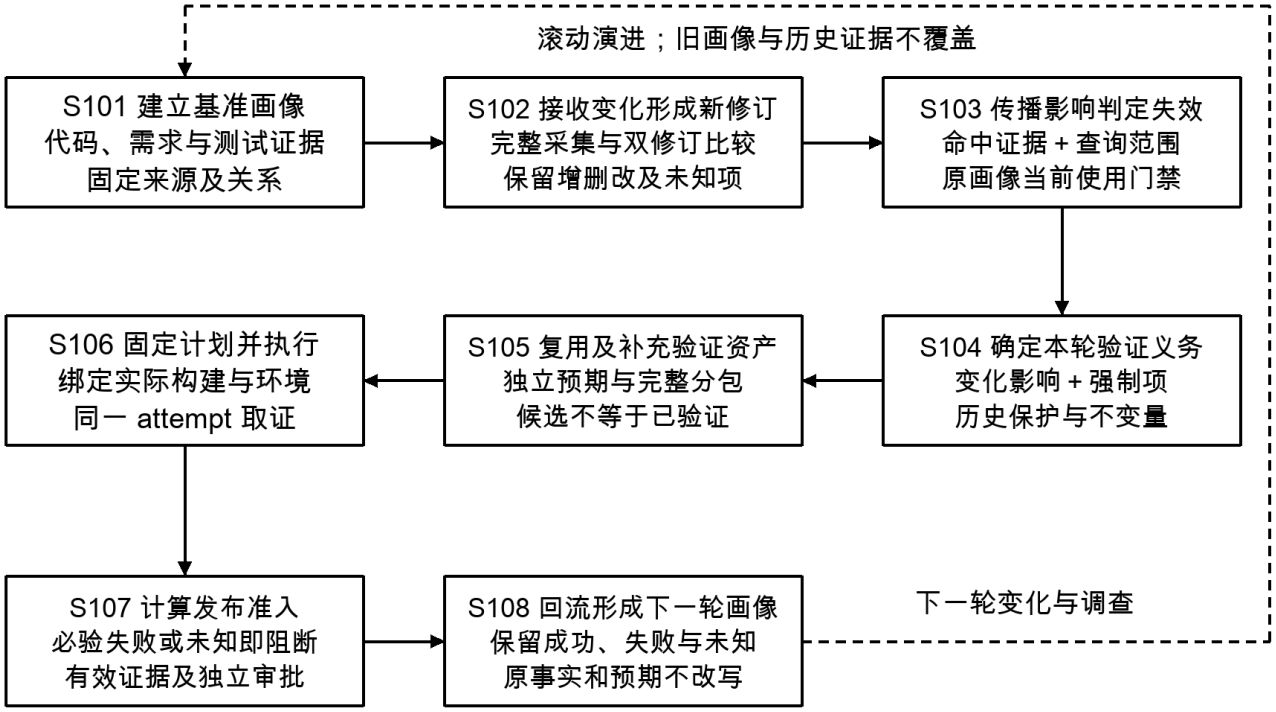



图1 滚动画像驱动集成验证和发布准入的总体闭环

图1中的处理链由来源采集、画像维护、影响分析、验证规划、执行与准入组件组成。数据库保存修订、关联和决定，对象存储保存原文、脚本及运行记录，检索索引提供可重建的查询视图。模型提出关系或测试候选；程序规则检查身份、完整性、状态和准入条件。

“集成验证”指接口、模块、数据状态、消息及外部依赖协同后的可观察行为验证。“发布准入”指目标版本满足进入发布流程的条件，生产部署是后续独立操作。

下表为方案的逻辑对象；组合字段不等同于当前已有同名 API 或数据库表。

| 对象组 | 决定性字段 | 数据约束 |
| --- | --- | --- |
| 来源与画像 | 来源绑定、修订、采集范围及完整性、内容摘要；画像 ID、父修订、来源清单、实体与关系修订 | 同一来源身份下比较修订；画像正文不可变，当前基线指针单独维护。 |
| 变化 | 双修订引用、文件/成员差异、范围证明、未决项、差异摘要 | 单侧缺失只有在另一侧同范围完整采集证明下才成为确定新增或删除。 |
| 影响与失效 | 旧/新关系图修订、命中证据、查询范围指纹、派生依赖、原因路径、未知范围、失效对象 | 业务影响与分析依据失效分别记录；当前使用失效绑定画像修订。 |

| 对象组 | 决定性字段 | 数据约束 |
| --- | --- | --- |
| 验收与义务 | 验收来源及批准修订、前置条件/操作/观察对象/预期；义务 ID、进入理由、子检查清单、全局规则、集合摘要 | 独立规范提供预期；义务及统计分母在本轮固定。 |
| 验证资产 | 用例/脚本修订、义务引用、步骤和断言映射、端点修订、适用决定、缺口 | 资产的使用范围与证据状态分别管理，每项义务具有资产映射或显式缺口。 |
| 执行计划 RunPlan | 画像/义务摘要、用例与脚本、全部来源修订、构建、环境、测试数据、期望证据清单、计划摘要 | 执行使用冻结输入，实际目标需与声明核验一致。 |
| 执行证据 | 计划、执行批次 attempt、用例/步骤/义务/判据、请求摘要、原响应、实际目标及环境记录、观察状态 | 同一目标及批次内逐项对账，失败、未执行和未知分别保留。 |
| 准入与反馈 | 目标、政策修订、义务及证据摘要、阻断项、批准、预期基线头、父画像与反馈引用 | 正式决定提交前重验依赖；观察和正式知识分别回流。 |

摘要使用固定编码、字段顺序和集合排序的规范序列化计算，用于检测内容或字段变化；业务语义正确性依靠独立判据和有效观察，实际目标身份依靠受控执行环境的核验。

## 2. S101 建立基准画像

**关键输入：** 项目来源绑定、代码提交、需求和历史测试修订、运行记录及批准规则。

固定全部来源的身份、范围、修订和原文定位，抽取模块、接口、数据对象、业务规则、用例及证据；用稳定实体 ID 连接调用、读写、实现和验证关系。代码观察、语义候选、正式规范和运行记录保留各自来源，关系保存端点身份及修订。

独立验收判据由批准的需求或规则修订确定，包含前置条件、操作、观察对象和预期。例如，实现观察是“当前返回200”，业务规范可以是“非所有者返回403且无记账”；两者冲突形成待验证问题，执行观察不能自动改写规范。

**输出及状态转换：** 形成基准画像 I0、来源组合清单及验收修订。画像区分待审候选、可用于本轮验证的工作画像和项目正式基线；候选经对应审阅成为工作画像，正式基线在发布及知识治理条件满足后推进。来源损坏、身份冲突和缺少判据分别形成拒绝接纳、待合并及待补判据状态。

## 3. S102 接收变化并形成新修订

**关键输入：** 同一来源绑定的旧修订 R0、新修订 R1、已有画像及代码、需求或测试变化事件。

保存 R1 的新清单并保留 R0。共同路径按内容摘要区分未变和修改；单侧路径结合另一侧的同范围完整目录判断新增或删除。读取失败、权限变化、排除规则变化导致的缺失记录为 unresolved。不同来源绑定不直接比较，多来源项目逐一固定各自修订。

文件改名可以通过内容匹配形成候选，实体身份和旧审阅的继承仍以身份及修订依据确定。文件变化、实体变化和仅索引变化分别记录。

**输出及状态转换：** 形成绑定双修订的 ChangeSet、新来源清单及画像候选；正在验证的画像快照保持不变。采集不完整保留未决项，并发刷新导致预期清单变化时重新计算。

## 4. S103 传播影响并判定失效

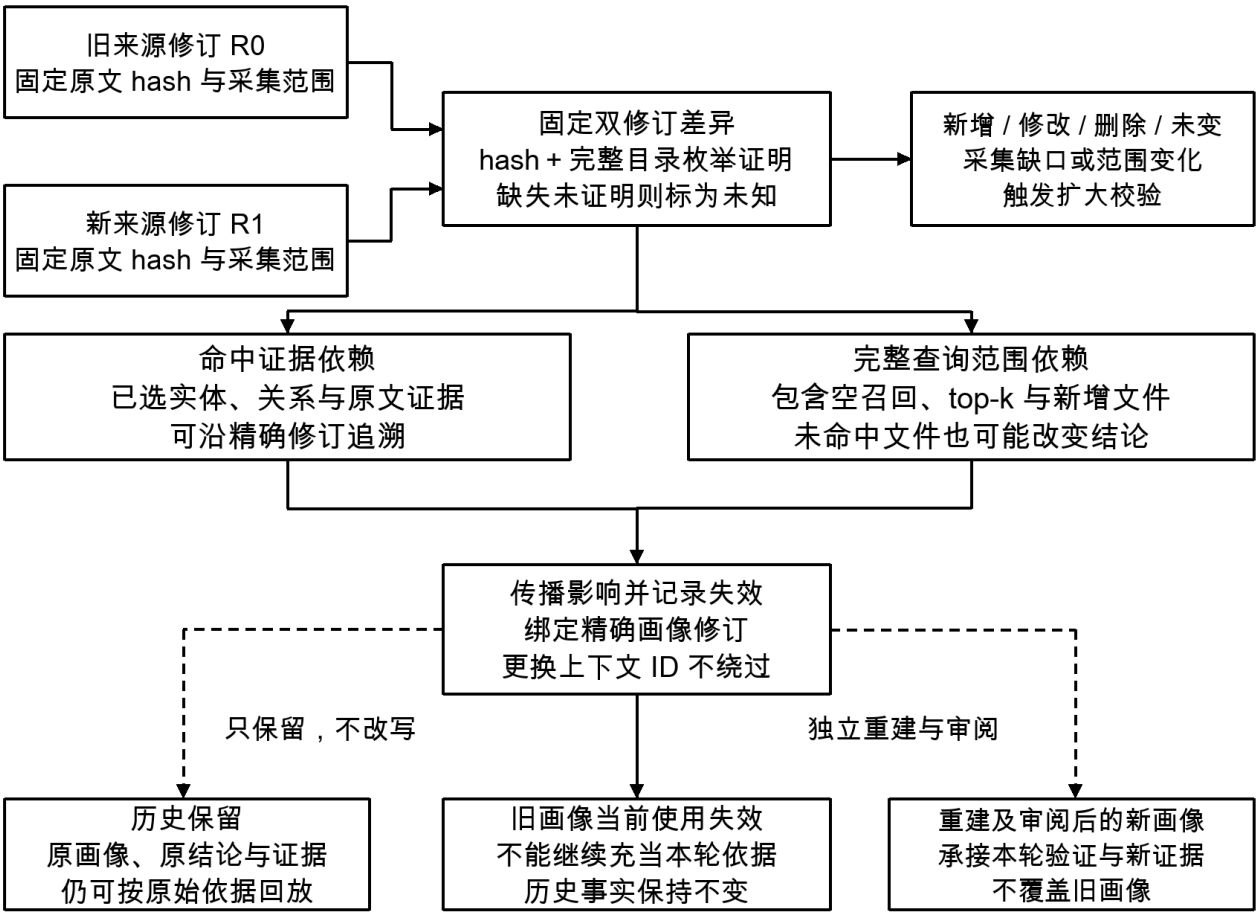



图2 双修订差异、依赖传播、失效与新画像

**关键输入：** ChangeSet、旧/新冻结关系图、派生依赖清单、查询范围及关系审阅记录。

图2区分两类依赖：系统依赖描述接口调用、数据读写及消息关系，回答“哪些业务可能受影响”；派生依赖记录上下文、测试方案和覆盖记录使用的来源及上游产物，回答“哪些分析依据需要重评估”。后者的闭包完整不等同于前者的业务影响完整。

查询范围依赖保存来源成员、范围指纹、输入及画像摘要，同时记录空结果与采集缺口。新增文件即使未被旧查询命中，也可能改变空召回或检索排序，因此与旧命中文件共同参与失效判断。

变化成员在旧、新图中分别传播：删除成员使用旧图端点，新增成员使用新图来源；被调用者变化反向追踪调用者，提供者变化追踪消费者，数据结构或消息契约变化关联读写者与订阅者。两图的影响结果按有依据的实体对应合并，原因路径保留所属修订。遍历遇到动态边、身份不明或节点/跳数截断时，扩大到模块的全部已知验证对象；模块边界不明时扩大到系统范围，并继续保留未知项。

历史关系按端点身份和修订分类：同版接受关系可继续使用；同版适用否定继续排除；端点变化进入重审；身份不明保留未知。否定筛选发生在检索候选截取之前。

**输出及状态转换：** 形成受影响业务、失效产物、原因路径和未知范围。失效绑定精确画像修订的当前使用，新建上下文 ID 或改变预算不恢复其适用性；历史内容不变。提交失效记录前复核来源与派生清单，预览后新增依赖触发重新计算。

### 核心算法一：差异与影响传播

以下伪代码描述方案算法；其实现范围统一列于“当前实现情况”。

```
def assess_change(before, after, old_image, new_image, dependencies, policy):
    verify_same_binding_and_image_sources(before, after, old_image, new_image)
    delta = []
    for path in sorted(before.paths | after.paths):
        if path in before.paths and path in after.paths:
            kind = "unchanged" if before.hash(path) == after.hash(path) else "modified"
        elif path in after.paths:
            kind = "added" if before.proves_absence(path, after.scope) else "unresolved"
        else:
            kind = "deleted" if after.proves_absence(path, before.scope) else "unresolved"
        delta.append((path, kind))
    roots = roots_from_hits_and_query_ranges(dependencies, before.binding)
    graph = verify_complete_registered_closure(roots, dependencies)
    invalid = conservative_derivation_impact(graph, delta)
    scope, unknown = propagate_in_separate_graphs(old_image, new_image, delta, policy)
    unresolved = [item for item in delta if item[1] == "unresolved"]
    if unknown or unresolved:
        scope = broaden_to_module_or_system(scope, policy)
    return freeze(before=pin(before), after=pin(after),
                  old_image=pin(old_image), new_image=pin(new_image),
                  graph_digest=graph.digest, dependency_digest=dependencies.digest,
                  delta=delta, scope=scope, invalid=invalid,
                  unknown=unknown, unresolved=unresolved)


def commit_invalidation(result):
    with project_transaction():
        revalidate_sources_images_and_dependency_digest(result)
        append_current_use_invalidation(result.old_image, result.invalid)
```

## 5. S104 确定本轮验证义务

**关键输入：** 目标工作画像、变更与影响评估、独立验收修订、历史故障及风险策略。

本轮必验集合定义为：

**O = 新增或变化行为义务 ∪ 直接影响义务 ∪ 策略要求的间接影响义务 ∪ 历史故障保护义务 ∪ 适用全局规则义务。**

每行保存稳定 ID、验收及修订、进入理由、适用目标、完整判据和子检查清单。按身份与修订去重，不按文本相似度合并不同权限或状态条件。全局规则保留唯一 ID，可以关联多个用例；规则副本不增加分母。

关键流程、公共契约和高风险失败模式按固定策略进入必验，其他候选进入可选集合。缺判据或依赖不明的行为仍作为显式缺口保留；范围扩大后未解决的关键未知继续传递。

**输出及状态转换：** 固定义务集合及摘要、可选回归集、排除理由与未知项。后续选测、生成和预算处理不能改变分母；空义务集合进入复核，不能直接生成通过结论。

## 6. S105 复用及补充验证资产

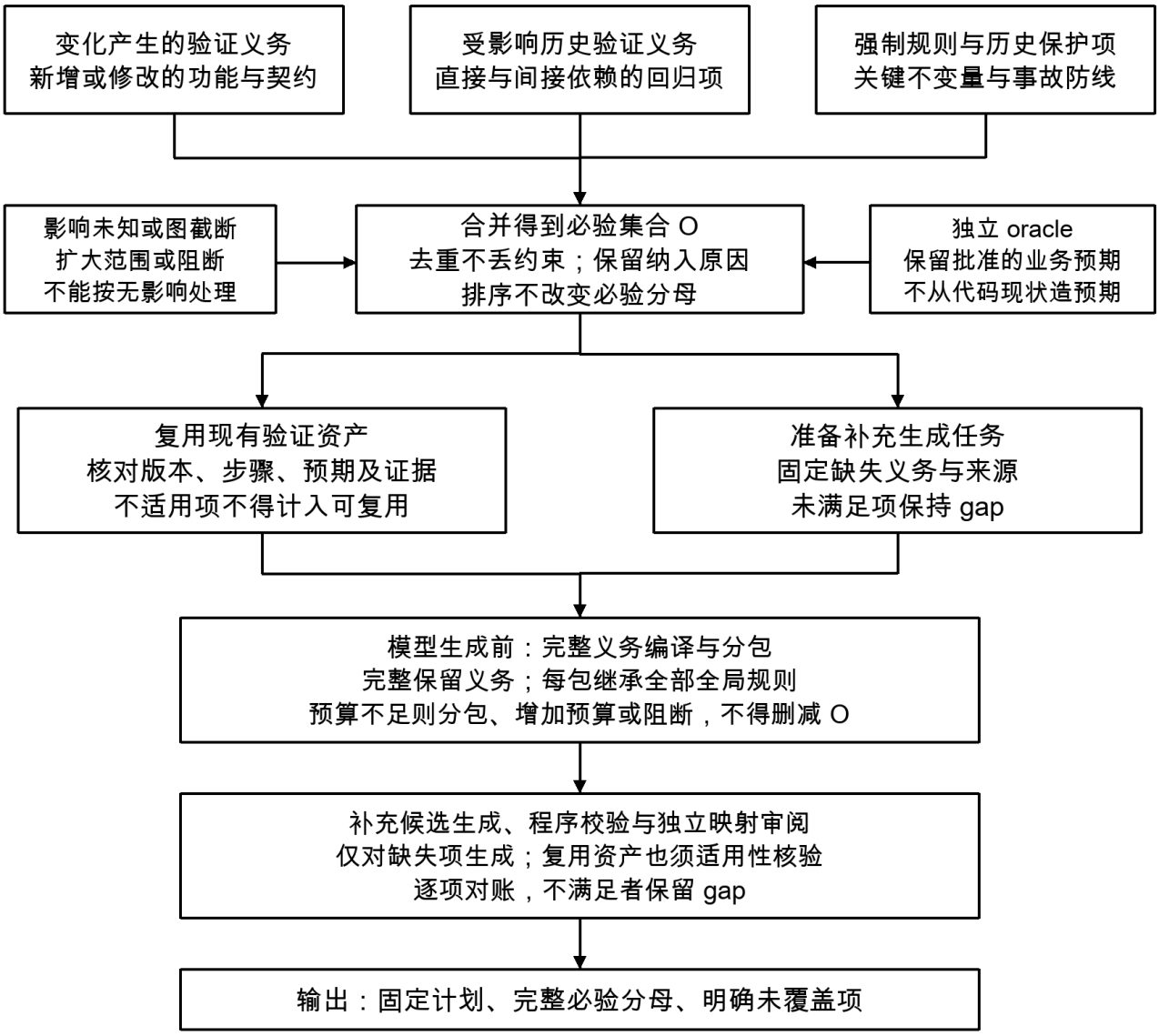



图3 必验集合、资产复用、补充生成及范围扩大

**关键输入：** 固定 O、历史用例和脚本、适用关系及支持证据。

先核验资产引用的验收修订、接口契约、步骤和数据前提，再确定复用、修改、重审或补充生成。复用的是测试资产；本轮必验项的通过依据仍须适用于本轮目标构建，旧构建的通过结果仅作历史参考。

生成上下文先放入完整义务和适用全局规则，再选择代码片段、历史失败等支持材料。将 O 分为互不重叠的非全局义务 A 与全局规则 G：每条 A 完整进入一个子包，每个子包继承全部 G；副本继续引用原 ID。风险分析扩展出的义务与原验收条件使用相同分包规则。

当前分包预算按规范 JSON 的 UTF-8 字节计量。模型调用另按固定模型、分词器和模板计算 token 预算，计入提示包装与输出预留；两种预算不使用通用比例换算。完整义务与全局规则超限时阻断，预算只裁剪可选材料和可选测试。

候选输出包含用例 ID、义务引用、原判据及摘要、前置条件、步骤、断言位置、证据引用和缺口。程序校验引用、唯一身份、步骤索引、判据一致性和义务对账；独立映射审阅判断步骤能否实际观察判据，脚本检查确认执行入口。预期被替换、义务遗漏、全局规则丢失及非法引用均拒绝接纳。

**输出及状态转换**：得到可复用或新增资产、义务—用例—步骤映射及显式缺口。每项义务都具有映射或缺口；模型调用失败保留未完成任务，重复用例不增加覆盖。

### 核心算法二：完整义务与验证集合编译

```
def compile_round(impact, image, approved_spec, policy, assets, budget):
    verify_approved_image_and_impact_binding(image, impact)
    O = union_by_identity_and_revision(
        changed_behavior(impact, approved_spec),
        direct_impact(impact, approved_spec),
        policy_required_indirect_impact(impact, policy),
        historical_failure_protections(impact, policy),
        applicable_global_rules(approved_spec, policy),
    )
    O = retain_explicit_gaps(O, impact.unresolved, impact.unknown,
                             approved_spec.missing_oracles)
    freeze_ids_denominator_reasons_and_checks(O)
    reusable, gaps = classify_assets(assets, O)
    A, G = separate_non_global_and_global_rows(O)
    packs = partition_whole_rows(A, global_rules=G, budget=budget,
                                 emit_global_only_pack_if_needed=True)
    require_each_primary_row_exactly_once(packs, A)
    require_all_global_rules_in_each_pack(packs, G)
    require_union_equals(packs, O)
    require_all_mandatory_content_fits_or_block(packs, budget)
    add_optional_evidence_if_fits(packs, budget)
    return freeze(image=pin(image), impact=pin(impact), policy=pin(policy),
                  O=O, reusable=reusable, gaps=gaps, packs=packs,
                  unresolved=impact.unresolved, unknown=impact.unknown)
```

## 7. S106 固定计划并执行

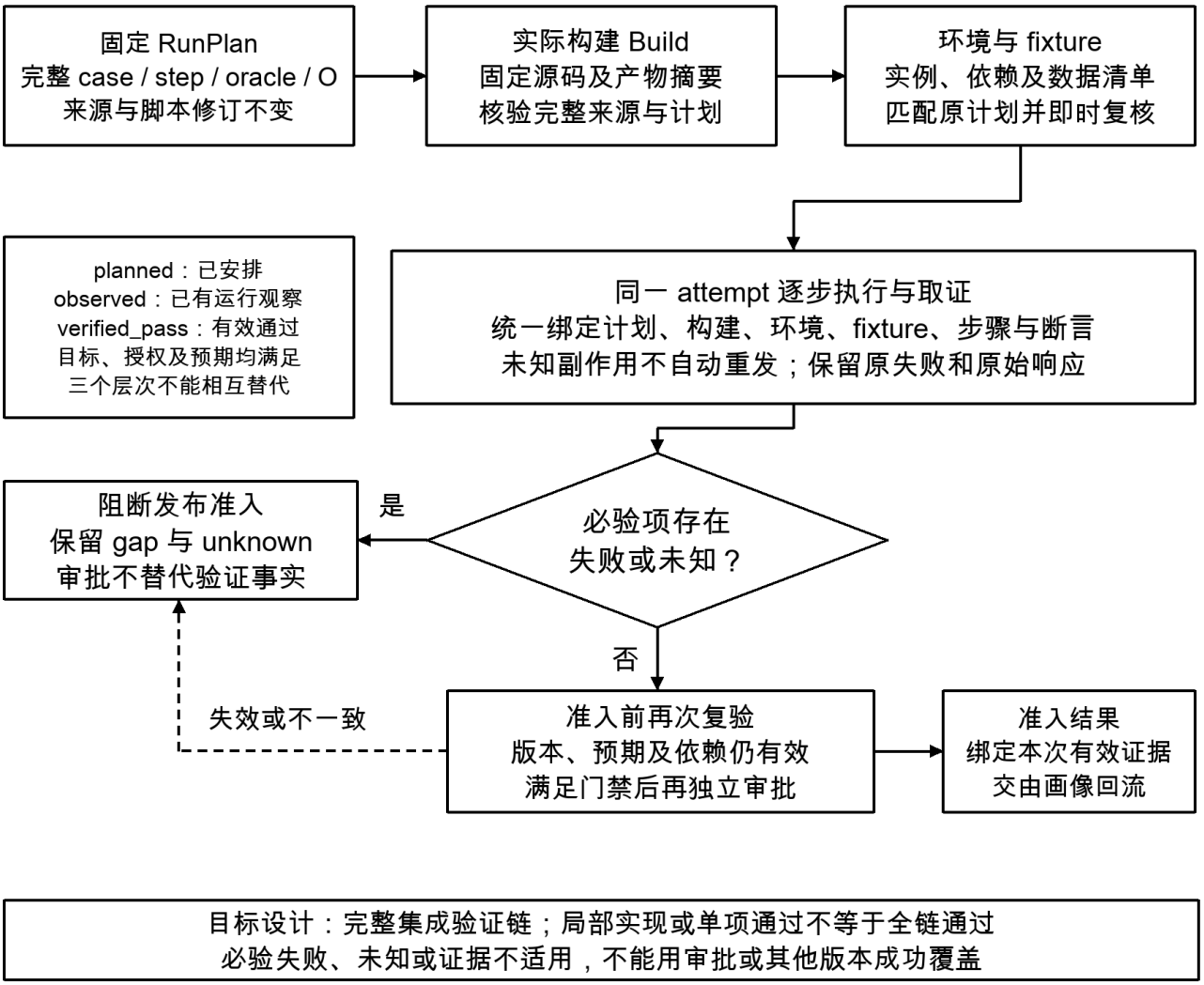



图4 RunPlan、目标构建、执行证据与准入状态

**关键输入：** 已审阅用例、脚本和映射，全部来源修订，构建、环境、测试数据、执行授权及期望证据清单。

RunPlan 固定义务摘要、用例/步骤/判据和全部目标输入。实际构建核验覆盖完整来源集合、构建定义、工具链及产物字节；环境核验覆盖实例、挂载产物、依赖和配置。执行前及关键请求前后检查实际目标，避免实例或构建在过程中被替换。目标身份由受控部署记录、产物检查或可信执行代理核验。

测试数据按用例隔离，固定前置状态、命名空间及准备/清理范围；身份通过受控认证适配器配置。派发前持久记录计划、批次、任务和执行所有者，恢复过程遵循原执行状态。请求可能已产生副作用却缺少回执时标 unknown，先按原请求查证，不直接自动重发。

保存实际请求摘要、原响应、日志和目标核验记录，接纳端从原响应计算断言。按计划逐项核对用例、步骤、义务、判据、构建、环境和 attempt；跨构建、跨批次记录不能拼成一轮完整证据。

| 状态 | 成立条件 |
| --- | --- |
| 计划覆盖 planned | 义务具有合法候选及计划映射。 |
| 资产可执行 automated | 脚本、映射审阅及执行入口满足要求。 |

| 状态 | 成立条件 |
| --- | --- |
| 原始观察 observed | 实际请求与原响应已核验并保存。 |
| 目标执行 executed_on_target | 观察同时具有原计划、实际目标、环境/数据及授权依据。 |
| 有效通过 verified_pass | 同一批次中，该义务的全部固定观察满足独立判据，且无未解决矛盾。 |
| 失败 failed | 有效观察违反对应验收判据。 |
| 未知或未执行 | unknown 表示操作或结果不确定；not_observed 表示未取得观察；not_executed 需要可信记录证明未执行。 |

**输出及状态转换：** 一次执行的原始证据集合、逐义务状态及未完成项。动作完成与业务断言分别计数；环境准备失败、业务失败、中断及清理异常分别记录。证据缺失、损坏、重复回执和目标错配保留诊断，不提升为有效通过。

## 8. S107 计算发布准入

**关键输入：** 固定 O、目标构建、完整证据集合、政策修订、关键缺口、批准及预期基线头。

准入以固定 O 为分母，逐项检查全部观察要求及适用性。必验失败、关键未知、未执行和证据缺失均阻止准入，综合分数与操作批准不能覆盖这些质量条件。

| 准入状态 | 判断条件 |
| --- | --- |
| blocked | 存在必验失败、关键依赖失效、目标错配、证据损坏或政策硬阻断。 |
| needs_evidence | 尚无确定失败，但存在缺判据、缺运行结果、未知或未确认的关键范围。 |
| awaiting_governance | 质量条件满足，批准、合并或预期基线头条件尚未满足。 |
| ready | 本轮全部必验项具有适用的有效通过证据，治理条件及提交前复核满足。 |

正式提交前，在受控事务中重验画像当前使用状态、验收批准、来源与目标适用性、证据集合摘要及预期正式头。批准撤回、画像失效、构建变化或竞争提交导致不一致时，拒绝旧评估并重新判断。未改变本轮目标及依赖的无关变化不凭时间戳使证据失效。

**输出及状态转换：** 保存绑定目标、义务、政策及证据的正式准入决定，或明确阻断/补证项。质量评估、操作批准、正式决定和生产部署分别记录。

### 核心算法三：证据接纳与准入判断

```
def assess_release(round_input, plan, attempt, evidence, policy):
    verify_frozen_inputs(round_input, plan, policy)
    verified = verify_complete_expected_evidence(
        plan, attempt, evidence, actual_build=True, environment_and_data=True,
        execution_authorization=True, original_responses=True,
    )
    rows = evaluate_all_required_checks(round_input.O, verified,
                                        preserve_failed=True, preserve_unknown=True)
    if integrity_or_target_error(verified) or any_required_failed(rows):
        status = "blocked"
    elif not round_input.O or critical_unknown_scope(round_input) or any_not_verified_pass(rows):
        status = "needs_evidence"
    else:
        status = "ready_for_review"
    return freeze(status=status, round=pin(round_input), plan=pin(plan),
                  target=plan.target, policy=pin(policy), attempt=attempt.id,
                  obligation_digest=round_input.O.digest,
                  evidence_digest=verified.collection_digest, rows=rows)


def commit_release(assessment, approval, expected_head):
    with project_transaction():
        revalidate_dependencies_targets_evidence_and_approval(assessment, approval)
        require_equal(current_head(), expected_head)
        require(assessment.status == "ready_for_review")
        require(no_hard_blockers_and_governance_complete())
        append_formal_ready_decision(assessment, approval, expected_head)
```

证据集合按计划的完整清单核验，不能由提交者挑选成功记录充当全部结果。any_not_verified_pass 包含未知、未观察、未执行、中断、缺判据和任一未满足的子检查。ready_for_review 是内部质量评估结果，经治理与提交复验后才形成正式 ready；等待阶段对应 awaiting_governance。接纳中的无权、归档损坏或不可用错误进入阻断或补证状态。

## 9. S108 回流并形成下一轮画像

**关键输入：** 运行观察、失败原因、资产适用决定、缺口处置及正式发布决定。

失败先进入本轮观察和风险候选，供下一轮调查与故障保护使用；确认后的关系、规则和测试资产经独立审阅进入版本工作画像。满足发布和知识治理条件后，固定父修订与候选摘要，以预期正式头的比较并交换推进基线。

正式头变化时进行三方比较和重新审阅；代码事实从实际合并提交重新提取，避免拼接出不存在的代码版本。反馈按来源事件和执行批次去重，限定更新范围，防止循环触发。

**输出及状态转换：** 形成下一轮可复用画像、资产及风险记录，保留旧画像和失败历史。历史复核使用原始输入、响应及当时规则；当前是否仍适用单独计算。归档损坏时停止复核，不以最新内容填补原始事实。

## 10. 关键技术难点与处理机制

| 技术难点 | 难点所在 | 处理机制与反例结果 |
| --- | --- | --- |
| 区分真实变化与观察缺失 | 单侧没有文件，既可能是增删，也可能是权限、过滤或采集失败。 | 同来源双修订加同范围完整性证明；证据不足保留 unresolved，避免把采集失败误当删除而缩小验证范围。 |
| 新增内容影响旧认知 | 旧命中列表没有新增文件，空查询也没有可追踪的命中文件。 | 查询范围与命中证据共同登记；新回调文件进入范围后触发原上下文重评估。业务图分别在旧、新修订传播。 |
| 历史关系与否定的适用性 | 同名实体或变化端点可能沿用旧接受/否定；新建上下文可能重新包装旧依据。 | 按身份及修订判断沿用、重审、未知；否定先于候选截取；失效绑定画像当前使用，独立保留历史正文。 |
| 快速选测与义务完整性 | 检索截断、模型预算和无可用脚本容易让必验行为消失。 | 先固定 O，再复用或生成；完整分包继承全局规则；依赖截断扩大范围，超限及缺资产形成显式阻断或缺口。 |
| 资产复用与结果适用性 | 可复用脚本并不意味着原结果描述当前构建。 | 资产审阅与证据接纳分离，RunPlan 固定实际目标和完整证据清单，逐项对账后才提升状态。 |
| 异步副作用与执行未知 | 网络超时不能说明操作未发生，重复发送可能造成重复退款。 | 先持久记录派发身份，接纳原始观察，截止仍不明保留 unknown；查证与新批次分开，重复回执按身份去重。 |
| 发布前失效与并发竞争 | 执行完成后批准、构建或基线头仍可能变化。 | 提交前重验固定依赖和预期头，决定在受控事务内保存；不一致重新评估。 |
| 反馈污染与历史复核 | 失败观察可能被误写成规范，更新可能覆盖旧解释依据。 | 观察、风险候选、正式知识分层；审阅后生成新修订，原输入及历史结论追加保留，损坏拒绝复核。 |

## 11. 当前实现情况

实现状态沿用2026年10月13日的源码与历史验证核查。下表中的“已实现组件”指有范围对应的实现及历史验证记录，不表示本轮重跑。定位路径属于主项目源码，专利分支仅保存文档。

| 环节 | 已有实现与代表定位 | 当前实现范围 |
| --- | --- | --- |
| 画像与独立验收 | 画像修订、来源追溯、验收批准；`image_revision.py`、`acceptance_specification.py`。 | **部分实现**。 基础对象和批准机制已有，全部语义关系及完整版本治理未贯通。 |
| 来源差异 | 双修订、采集 inventory、unresolved、代码成员差异；`source_delta.py`。 | **已实现组件**。 支持有界来源及成员比较；自动变化调度与完整增量重建为设计。 |
| 影响与失效 | 静态影响、查询范围、派生闭包、当前使用失效；`derivation_impact.py`、`context_invalidations.py`。 | **部分实现**。 当前对固定上下文派生集合采取保守失效，保留 impact_complete=false；全项目精确影响分析与最小回归集合未实现。 |

| 环节 | 已有实现与代表定位 | 当前实现范围 |
| --- | --- | --- |
| 义务与资产 | 独立判据、完整分包、生成任务、候选校验、映射及脚本审阅；`context_partition.py`、`generation_jobs.py`。 | **部分实现**。 约束生成组件已有；由变更自动形成完整义务并驱动资产复用的组合为设计。 |
| 计划与构建 | prepared RunPlan、完整来源绑定、原生构建；`run_plan.py`、`native_builds.py`。 | **已实现组件**。 构建验证集中在 Java/Maven 等限定适配器；prepared 不授予执行通过。 |
| 执行与证据 | 目标 HTTP 执行、数据生命周期、原响应核验及持久观察登记；`owned_target_http.py`、`run_plan_observations.py`。 | **部分实现**。 已取得组件级实际观察；正式业务覆盖和完整执行生命周期尚未贯通。 |
| 准入与回流 | 确定性质量评估、未知阻断、正式决定、批准和基线提升；`release_readiness.py`、`governance.py`。 | **部分实现**。 完整逐义务证据消费与自动反馈演进为设计；既有产品保留 conditional 等兼容状态。 |

整体方案包含现有组件和可实施的完整设计，**自动增量、正式执行、业务覆盖、发布准入及画像回流尚未全部贯通**。本文的准入状态表描述目标方法，未将其视为全部已上线 API 状态。

## 12. 订单退款两轮变更实施例

以下为构造实施例，用于展示数据、处理规则和状态转换，不属于真实业务实验结果。

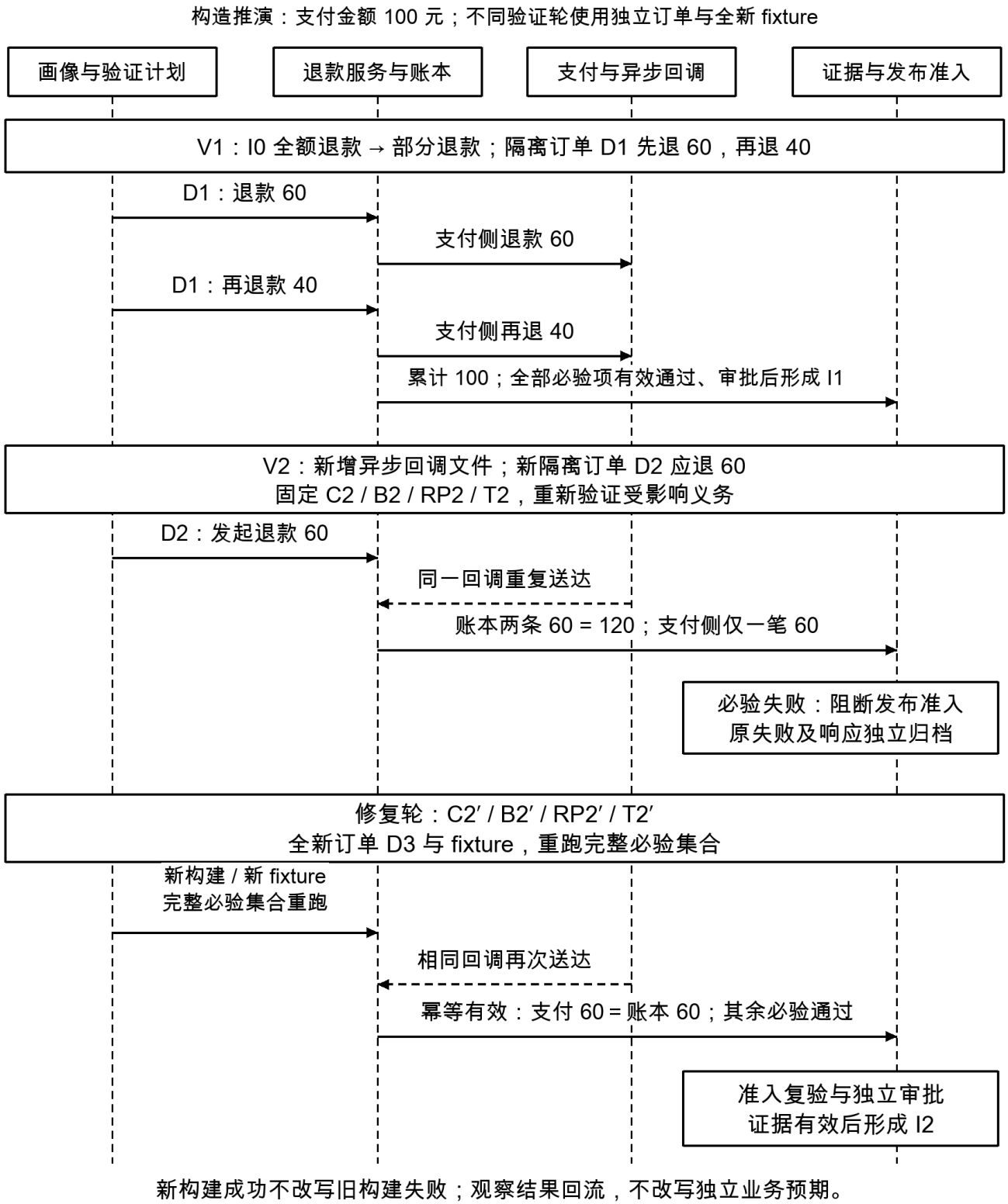



图5 两轮变更、失败修复、发布判断与画像回流

### 12.1 初始画像与测试环境

I0 包含订单服务、退款服务、支付适配器、账本及权限关系，初始功能为全额退款。独立规范规定：只有订单所有者可申请；同一业务退款只能记账一次；成功退款总额不超过实付金额。

测试环境使用隔离订单、支付模拟服务、只读账本查询和受控身份。支付交易与账本按 business_refund_id 核对；用例间隔离数据，同一用例的连续步骤共享其原订单。每个订单默认实付100元，所有者为 U1，非所有者为 U2。异步等待设置30秒截止和有界轮询，截止后仍不明确则保留未知。清理仅处理本次准备记录中的资源。

### 12.2 第一轮：从全额退款增加部分退款

AI Coding 任务形成 C1 和需求修订 A1。经批准的 A1 明确金额、幂等和权限预期：金额为正且不超过剩余额度；同键同金额返回同一业务标识；同键不同金额拒绝；未认证返回401，非所有者返回403，均不得产生支付或记账。

差异沿订单、支付、账本和权限关系传播，历史权限与幂等用例进入适用性检查，新增额度、状态和并发验证。O1 固定以下六行：

| 义务 | 验证动作及完整观察 |
| --- | --- |
| O1-01 合法部分退款 | 对隔离订单 D1 退60元；支付成功一笔60元，账本一笔60元，订单为部分退款。 |
| O1-02 剩余额度与状态 | 同一用例内继续退40元；累计100元、两笔业务退款，订单转为全额退款完成。 |
| O1-03 非法金额与超额 | 独立前置状态分别输入0、负数及超过剩余额度的金额；拒绝且无新增支付或记账。 |
| O1-04 幂等与冲突 | 同键同金额返回原业务标识且仅记一次；同键不同金额拒绝且无新增副作用。 |
| O1-05 权限与认证 | U2 请求返回403，无认证返回401；支付和账本均无新增记录。 |
| O1-06 并发与总额规则 | 对另一独立订单同时申请两笔80元；恰一笔成功80元，另一笔因余额不足被拒绝，支付与账本按业务标识一致，总额80元。 |

每行保存完整子检查清单，六行不等于六条请求。O1-03 的三种边界、O1-05 的两类身份分别取得证据。金额全局规则使用 O1-06 的唯一身份，并包含其他相关用例的金额核对；分包复制引用不增加第七项义务。并发两笔均被拒绝也不满足 O1-06。

复用权限和幂等资产，补充额度及并发用例；候选把403替换成200时拒绝，缺少支付或账本观察时保留映射缺口。工作画像 W1 获准用于验证后，RP1 固定 W1、O1、C1/B1、环境 E1、数据 F1及脚本，在批次 T1 执行。

在六行全部观察有效通过、无关键缺口且治理条件满足的构造条件下，形成 B1 的准入决定；发布流程及知识审阅完成后，资产与关系进入正式画像 I1。

### 12.3 第二轮：异步回调引入重复记账

新任务增加 payment_callback_handler，形成 C2。该文件未出现在旧命中列表，却属于被冻结的来源查询范围，因此触发旧上下文重评估。新增消息、回调、退款业务 ID 与账本关系进入影响分析，形成工作画像 W2。

新规范 A2 将原行为调整到异步场景。O2 保留前六项要求，并增加两项，固定为八行：

- **重复回调**： 同一成功事件多次到达，支付交易和成功账本各保留原一笔。
- **乱序与终态重放**： 同一退款先收到 SUCCEEDED(seq=2)，再收到 PROCESSING(seq=1)，最后重放原成功事件；退款保持成功、订单保持相应部分退款状态，账本仍为原一条，不新增支付。

RP2 固定 W2、O2、C2/B2、E2、独立数据 F2 和批次 T2。对隔离订单 D2 退60元，模拟支付只有一笔60元交易；重复投递成功事件后，错误实现生成两条60元成功账本，总额120元。原响应、事件 ID、交易和账本共同形成失败证据。

结果违反幂等及金额要求，B2 进入 blocked，失败观察进入风险候选；原规范保持“一次记账”，失败候选不晋级为正式 I2。

### 12.4 修复后的重新判断

修复形成 C2′/B2′，重新计算影响和资产适用性，创建 RP2′，使用新订单 D3、数据 F2′及批次 T2′执行。只有 O2 八行的全部要求由 B2′对应证据满足，且治理与提交复核通过，才能形成 ready；后续完成知识治理再形成正式 I2。

| 内容 | 下一轮的使用方式 |
| --- | --- |
| 未变规范、历史用例及脚本 | 核验修订与适用性后复用，异步入口和观察步骤重新适配。 |
| B1/T1 成功证据 | 保留为历史说明和选测依据，不计入 B2 或 B2′通过分子。 |
| B2/T2 失败证据 | 保留修复依据和故障保护理由；不覆盖、不拼入 T2′。 |
| B2′/T2′ 新证据 | 按 RP2′逐项满足本轮完整义务，独立产生准入结论。 |

# 三 技术保护点

## 1. 核心技术保护组合

核心组合是一种计算机执行的变更驱动集成验证与发布准入方法：

1. 以版本化系统画像固定来源、关系和验收依据，根据同来源双修订及采集完整性识别变化。
2. 根据系统关系、命中证据及查询范围依赖确定影响与失效，将结果转化为具有稳定身份和独立判据的本轮必验集合。
3. 在保持必验集合完整的条件下复用或补充资产，形成绑定画像、义务、来源、构建、环境及数据的执行计划。
4. 接纳与计划、实际目标和批次相符的观察，按义务判定通过、失败及未知，并在提交前重新检查依赖和批准，形成发布准入结果。
5. 分层保存运行观察与经确认的知识，生成后续画像修订，同时保留原始依据及历史结论。

上述环节通过数据绑定相互制约：影响与失效决定验证范围，固定集合约束生成和执行，目标证据约束准入，反馈审阅约束后续规范的变化。

## 2. 保护点与技术限定

| 保护点 | 具体技术限定 | 对应步骤及附图 | 作用机制 |
| --- | --- | --- | --- |
| P0 连续绑定的主组合 | 画像变化及失效、本轮必验集合、目标证据、准入复验和分层反馈连续关联。 | S101—S108；图1、4、5 | 使每轮验证范围和准入结论具有同一组可追溯依据。 |
| P1 采集完整性约束变化 | 同来源双修订比较；单侧缺失依赖同范围完整采集证明，否则保留未决。 | S102；图2 | 防止观察缺失被当成确定增删，进而错误缩小范围。 |
| P2 查询范围参与失效 | 同时记录命中证据、空结果及来源范围，新资料也能触发派生产物重评估。 | S103；图2 | 覆盖仅追踪旧命中无法发现的变化。 |
| P3 按修订处理关系适用性 | 端点身份、修订和来源决定沿用、重审或未知；适用否定先于候选截取。 | S101、S103、S105；图2、3 | 控制旧审阅误用及否定关联重新引入。 |
| P4 当前使用失效与历史分离 | 失效绑定精确画像修订，新建上下文不恢复适用性；历史正文和决定保持不变。 | S103、S108；图2、5 | 阻止过期依据重新包装，并支持原版本复核。 |
| P5 必验集合完整保持 | 固定义务及子检查清单，完整条件分包，全局规则继承，缺口保留；未知扩大范围，超限阻断。 | S104、S105；图3 | 预算、排名和资产缺失不静默删除必验要求。 |
| P6 计划与目标证据绑定 | 计划固定来源、构建、环境、数据和期望观察；接纳时核验实际目标及同批次逐项证据。 | S106；图4 | 防止旧构建、错误目标和跨批次结果被计为本轮通过。 |
| P7 硬条件与提交复验 | 失败及关键未知阻断；正式提交重验依赖、批准、证据摘要和预期头。 | S107；图4、5 | 限制陈旧结论、审批覆盖质量缺口及竞争提交。 |
| P8 分层反馈与修订演进 | 观察先进入风险候选，规则和资产经审阅形成新修订；固定父修订、去重并保留历史。 | S108；图1、5 | 避免执行结果污染正式预期和反馈循环，支持连续演进。 |

## 3. 保护对象

方法保护处理步骤及数据约束；系统保护画像管理、影响分析、义务编译、执行、证据核验、准入和反馈组件之间的输入输出关系；电子设备和计算机可读存储介质承载同一方法。技术保护主题不依赖特定模型品牌、语言或存储产品，通用模型、图谱和人工审批本身不单独构成保护点。

# 四 技术效果与业务效果

## 1. 技术机制与最终效果

| 原有问题 | 技术机制 | 效果表现 | 评价指标 |
| --- | --- | --- | --- |
| 系统理解滞后且重复分析 | 画像保存适用知识，双修订差异与依赖定位重评估对象。 | 后续变更复用已有认知，集中处理变化相关部分。 | 重分析对象/字节数、画像更新时间、人工定位时间。 |
| 重复生成和无关回归成本高 | 先固定必要行为，再复用适用资产及补充缺口。 | 缩短从变更到可信反馈、再到准入判断的路径。 | 变更到首个可信反馈及准入耗时、执行资源量、资产复用率。 |
| 新增内容及采集缺口被遗漏 | 查询范围依赖、完整采集证明和未知传播共同控制影响。 | 新文件及空召回变化进入重评估，采集不全保持显式缺口。 | 真实变化漏识别数、错误增删判定数、应失效未失效数。 |
| 预算和选测丢失必验要求 | 固定 O、完整分包、全局规则继承、逐项对账。 | 必验要求持续存在，未完成部分可见，重复用例不抬高覆盖。 | 相对于独立验收集合的遗漏率、静默丢义务数、重复计数数。 |
| 错误证据导致错误放行 | 实际目标绑定、完整观察清单、硬条件和提交前重验。 | 错配、过期、不完整及未知证据不进入有效通过，缺口对应明确阻断。 | 错误证据接纳率、错误准入数、正常结果误拒率、缺陷检出及漏检。 |
| 历史结论难以解释 | 原始输入、响应和决定保留，当前适用性独立维护。 | 可追溯失败修复过程，并复核当时结论使用的依据。 | 复核一致率、归档损坏检出率、问题定位耗时。 |

这些机制使开发者更早获得跨模块集成反馈，减少重复阅读、重复生成和人工核对版本的工作；发布人员能够直接定位未满足的义务及证据缺口。效果依赖实际系统、变化分布和执行环境，当前没有端到端实测数据或提升比例。完整归档、全局规则复制、重验和保守扩大范围带来的资源成本同样计入评价。

## 2. 效果评价口径

评价使用相同的代码变化、验收要求、测试资产、环境和资源条件，对照全量回归、常规影响选测、本方案，以及分别移除查询范围依赖、义务保持或目标/提交复验的消融组。耗时与缺陷检出、遗漏、错误证据接纳、错误准入同时报告；采集、分析、生成、数据准备、执行及治理等待分段统计，初始化与后续轮次分别计算。新增文件、采集缺口、依赖截断、预期改写、目标更换、未知回执、跨批次拼接、归档损坏、批准撤回及竞争提交作为反例，检查其是否产生规定的重评估、范围扩大、补证或阻断结果。

# 参考资料

R1. Optimized regression testing through dependency graph analysis and selective test execution，US20260072811A1，2026。公开文本

R2. Predictive Test Selection，2018/2019。论文及版本记录

R3. 《人工智能集群交付事件的管理方法、装置、设备和介质》，CN122044758A，2026。公开文本

R4. in-toto: Providing farm-to-table guarantees for bits and bytes，USENIX Security，2019。会议论文

R5. 《一种面向软件V字开发流程的RAG检索方法及系统》，CN121996618A，2026。公开文本

R6. 《测试用例生成方法、装置、电子设备和存储介质》，CN117971698A，2024。公开文本

R7. The Oracle Problem in Software Testing: A Survey，IEEE Transactions on Software Engineering，2015。论文 DOI